プログラミングワークショップ 2

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

目次

- テーマ　配列
- 例題
  - 配列の復習
- 課題
  - 配列を使って複数のブロックを配置する

配列の復習

- 教科書 P165 Example 10-9
- 配列の作り方
  - データ型の定義、配列を宣言
  - newキーワードを使って配列を生成し、長さ（要素数）を定義
  - 各要素に値を代入
- 配列の長さ 配列名.length
  - 配列と繰り返しを組み合わせて使う

Example 11-9

```
int num = 60;
int[] x = new int[num];
int[] y = new int[num];

void setup(){
  size(240, 120);
  noStroke();
}

void draw(){
  background(0);
  for(int i = x.length-1 ; i>0 ; i--){
    x[i] = x[i-1];
    y[i] = y[i-1];
  }
  x[0] = mouseX;
  y[0] = mouseY;
  for(int i=0 ; i<x.length ; i++){
    fill(i*4);
    ellipse(x[i], y[i], 40, 40);
  }
}
```

- データ型の定義、配列を宣言
- newキーワードを使って配列を生成し、長さ（要素数）を定義

オブジェクトの配列

- 教科書 P167 Example 11-10
- オブジェクトの配列の作り方
  - データ型の定義、配列を宣言
  - newキーワードを使って配列を生成し、長さ（要素数）を定義
  - 各オブジェクトをnewキーワードを使って生成

Example 11-10 メインタブ

```
JitterBug[] bugs = new JitterBug[33];

void setup() {
  size(240, 120);
  for (int i = 0; i < bugs.length; i++) {
    float x = random(width);
    float y = random(height);
    int r = i + 2;
    bugs[i] = new JitterBug(x, y, r);
  }
}

void draw() {
  for (int i = 0; i < bugs.length; i++) {
    bugs[i].move();
    bugs[i].display();
  }
}
```

- データ型の定義、配列を宣言
- newキーワードを使って配列を生成し、長さ（要素数）を定義

- 各オブジェクトをnewキーワードを使って生成（繰り返しと組み合わせて使う）

- 各オブジェクトをメソッドを繰り返しと組み合わせて使う

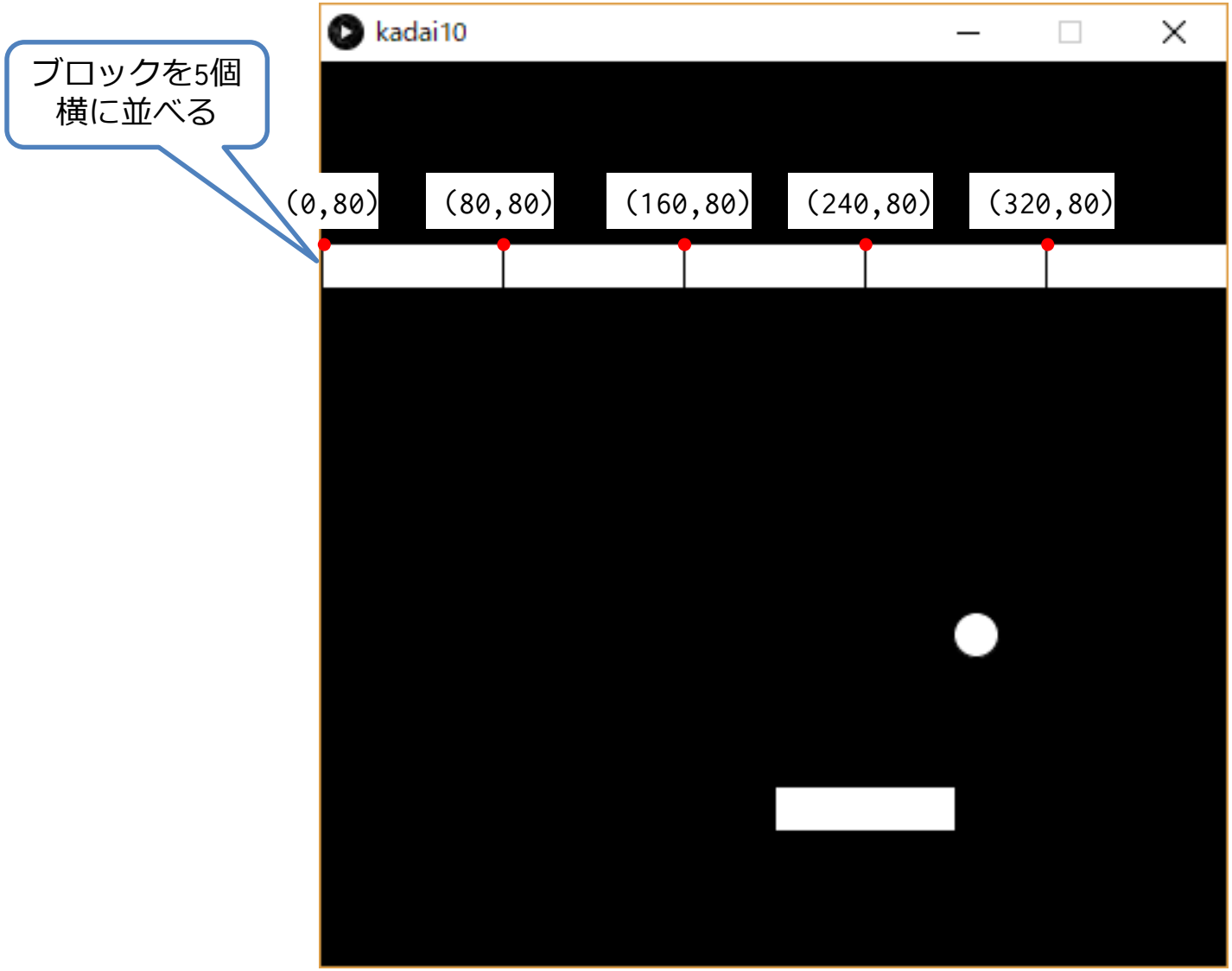
JitterBugクラス

```
class JitterBug {

  float x;
  float y;
  int diameter;
  float speed = 2.5;

  JitterBug(float tempX, float tempY, int tempÐiameter) {
    x = tempX;
    y = tempY;
    diameter = tempÐiameter;
  }

  void move() {
    x += random(-speed, speed);
    y += random(-speed, speed);
  }

  void display() {
    ellipse(x, y, diameter, diameter);
  }
}
```

課題 Kadai_11

- 課題10をもとに、複数のブロックを配置したプログラムを作成する
  - メインタブだけ変更、各クラスは変更しない

ブロックを複数配置する



メインタブ

```
Ball  ball;
Bar   bar;
Block block;
Wall  topWall, bottomWall, leftWall, rightWall;

void setup() {
  size(400, 400);
  ellipseMode(RADIUS);
  ball = new Ball(110, 60, 0.5, 0.3);
  bar = new Bar(width/2, height*0.8);

  block = new Block(40, height*0.2);

  topWall    = new Wall(0, 0, width, 0);
  bottomWall = new Wall(0, height, width, 0);
  leftWall   = new Wall(0, 0, 0, height);
  rightWall  = new Wall(width, 0, 0, height);
}
```

変更：Block型の配列blocks，要素数5

変更：繰り返しを使って各配列要素を生成

メインタブ

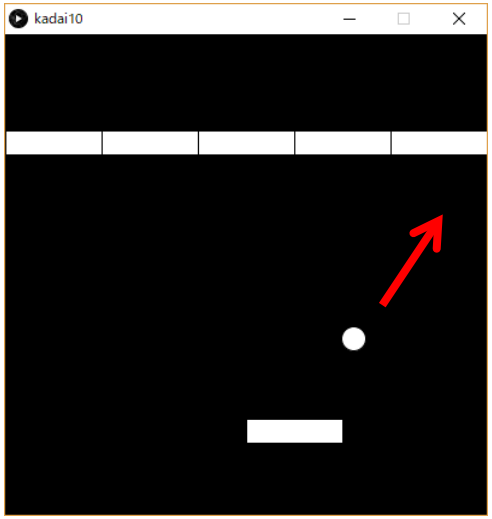
```
void draw() {
  background(0);

  ball.move();
  bar.move();

  if (leftWall.isHitRightSide(ball) || rightWall.isHitLeftSide(ball)) {
    ball.flipXdir();
  }
  if (topWall.isHitBottomSide(ball) || bottomWall.isHitTopSide(ball)) {
    ball.flipYdir();
  }

  if(bar.isHitTopSide(ball) || bar.isHitBottomSide(ball)){
    ball.flipYdir();
  }
  if(bar.isHitRightSide(ball) || bar.isHitLeftSide(ball)){
    ball.flipXdir();
  }
  if(bar.isHitCorner(ball)){
    ball.flipXdir();
    ball.flipYdir();
  }
```

メインタブ

```
  if(block.isHitTopSide(ball) || block.isHitBottomSide(ball)){
    ball.flipYdir();
    block.remove();
  }
  if(block.isHitRightSide(ball) || block.isHitLeftSide(ball)){
    ball.flipXdir();
    block.remove();
  }
  if(block.isHitCorner(ball)){
    ball.flipXdir();
    ball.flipYdir();
    block.remove();
  }

  ball.display();
  topWall.display();
  bottomWall.display();
  leftWall.display();
  rightWall.display();
  bar.display();
  block.display();
}
```

変更：繰り返しを使って
ボールとブロックの衝突判定

変更：繰り返しを使ってブロックを表示

動作確認

- ボールが壁で反射するかどうか
- ボールが反射板で反射するかどうか
- ボールがブロックで反射するかどうか
- ボールがブロックに衝突したらブロックが消えるか
- ブロックのあった場所をボールが通過しても反射しないか